

Report for Real-Time Face-
Recognition

Student Name : **L.Sharon Danish Laus**

Student ID : **25249347**

Revision Timestamp: 08/10/2025 20:48:28

Contents

Introduction	2
Training Script	3
Facial Recognition System	3
Experience	4
Performance	4
HOG	5
Conlcusion	5
References	5

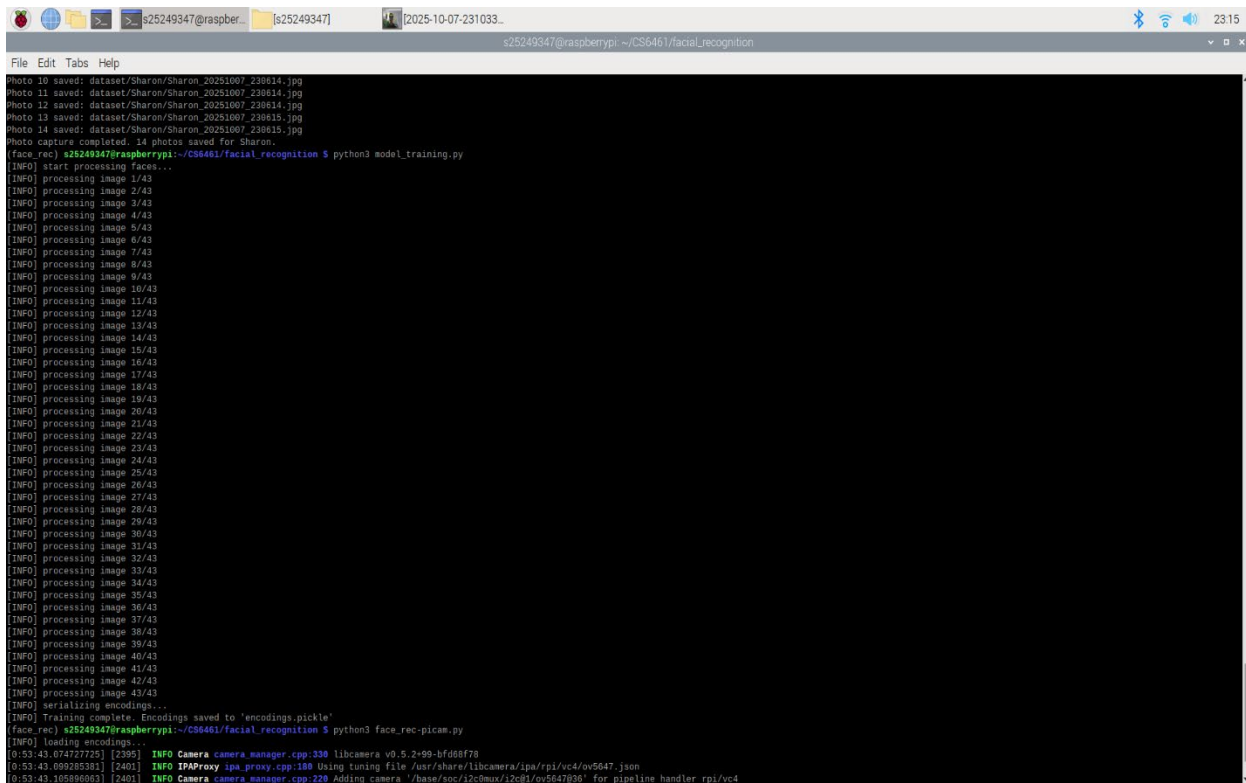
Figure 1: Training Script Running.....	3
--	---

Figure 2: Face Recognition Running.....	3
---	---

Introduction:

The report outlines the process of designing and developing a real time face recognition system that can be viewed as a complete computer vision application on the Raspberry Pi platform, which encompasses data acquisition, model training and inference on the Pi camera. The current methodology is based on the CS6461 Lab-3 specification, which prioritizes the competent usage of a Python virtual environment, deployment of OpenCV and the face-recognition library (using dlib) and exploitation of a version-managed source code that is obtained in a publicly accessible Git repository. A personalized set of headshots was gathered to train a facial encoding pipeline based on the histogram of oriented gradients (HOG) to generate a saved model representation (encodings.pickle) to be used in the prediction of identities in real-time camera streaming. This system is tested in real time through face recognition and annotation of identified persons extracted out of the video feed; this is later applied in the case of multiple persons by adding another subject and retraining again to test robustness. The laboratory assignment also requires recording of training and inference operations in the form of screenshots with a brief analysis of the performance and reason as to why HOG suits face detection finally contextualizing the implementation to the major concepts and best practices of embedded vision workflows on Raspberry Pi OS.

Training Script:



```
Photo 10 saved: dataset/Sharon/Sharon_20251007_230614.jpg
Photo 11 saved: dataset/Sharon/Sharon_20251007_230614.jpg
Photo 12 saved: dataset/Sharon/Sharon_20251007_230614.jpg
Photo 13 saved: dataset/Sharon/Sharon_20251007_230615.jpg
Photo 14 saved: dataset/Sharon/Sharon_20251007_230615.jpg
Photo capture completed. 14 photos saved for Sharon.
(face_rec) s25249347@raspberrypi:~/CS6461/facial_recognition $ python3 model_training.py
[INFO] start processing faces...
[INFO] processing image 1/43
[INFO] processing image 2/43
[INFO] processing image 3/43
[INFO] processing image 4/43
[INFO] processing image 5/43
[INFO] processing image 6/43
[INFO] processing image 7/43
[INFO] processing image 8/43
[INFO] processing image 9/43
[INFO] processing image 10/43
[INFO] processing image 11/43
[INFO] processing image 12/43
[INFO] processing image 13/43
[INFO] processing image 14/43
[INFO] processing image 15/43
[INFO] processing image 16/43
[INFO] processing image 17/43
[INFO] processing image 18/43
[INFO] processing image 19/43
[INFO] processing image 20/43
[INFO] processing image 21/43
[INFO] processing image 22/43
[INFO] processing image 23/43
[INFO] processing image 24/43
[INFO] processing image 25/43
[INFO] processing image 26/43
[INFO] processing image 27/43
[INFO] processing image 28/43
[INFO] processing image 29/43
[INFO] processing image 30/43
[INFO] processing image 31/43
[INFO] processing image 32/43
[INFO] processing image 33/43
[INFO] processing image 34/43
[INFO] processing image 35/43
[INFO] processing image 36/43
[INFO] processing image 37/43
[INFO] processing image 38/43
[INFO] processing image 39/43
[INFO] processing image 40/43
[INFO] processing image 41/43
[INFO] processing image 42/43
[INFO] processing image 43/43
[INFO] serializing encodings...
[INFO] Training complete. Encodings saved to 'encodings.pickle'
(face_rec) s25249347@raspberrypi:~/CS6461/facial_recognition $ python3 face_rec-picam.py
[INFO] loading encodings...
[0:53:43.074727725] [2395] INFO Camera camera_manager.cpp:330 libcamera v0.5.2+99-bfd08f79
[0:53:43.092325321] [2401] INFO IPAPProxy ipa_proxy.cpp:180 Using tuning file /usr/share/libcamera/ipa/rpi/vc4/v5647.json
[0:53:43.105869063] [2401] INFO Camera camera_manager.cpp:220 Adding camera '/base/soc/l2c0mux/l2c01/ov5647@306' for pipeline handler rpi/vc4
```

Figure 1: Training Script Running

Face Recognition System:

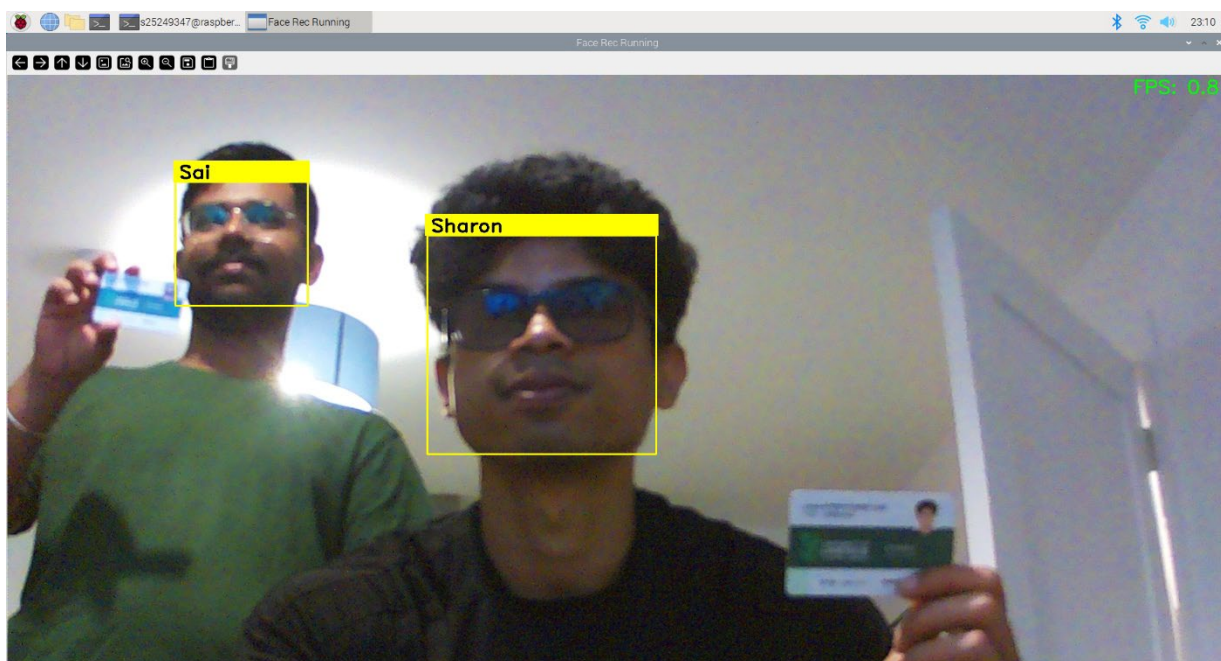


Figure 2: Face Recognition Running

Experience:

A fully assembled face -recognition pipeline was running on Raspberry Pi platform. The training phase created credible encodings and the live application identified two overlapping subjects correctly as indicated by the screenshots provided as well as according to the requirements of the lab brief. Although the performance was reasonable with moderate viewpoints when using indoor light, the system had lower frame rate and a higher sensitivity to large pose variations, results which are in line with the nature of a HOG based frontal face detector.

Procedural Experience: There was a prescription of the CS6461 Lab -3 sequence that was followed. This involved formation of a virtual setting, downloading of OpenCV and face-recognition library, obtaining 15-20 headshots of each subject, running of model training.py to create the face embeddings, and actual implementation of the real-time recognizer thus meeting the deliverables outlined in the lab brief.

There were no critical impediments experienced other than time-consuming installation and compilation stages. The mitigations proposed in the tutorial (temporary increase of the swap space and forcible re-installation of NumPy/OpenCV to avoid camera-library failure) created a clean install and run environment.

Performance:

Performance Evaluation: The live application was able to identify a number of faces in one frame and the on-screen counter showed an approximate throughput of 0.8 fps at the time of the demonstration. These findings show that the pipeline is working though with low throughput on the Raspberry Pi.

Detector Stability: The system was considered to be stable on detecting frontal and moderately tilted yaw/pitch angles in normal indoor lighting condition. The accuracy and recall dropped however, when faces had huge pose differences or had undergone backlighting-behaviour which is expected of dlib HOG based frontal face detector using a linear classifier, image pyramid and sliding-window search strategy.

Improvement Opportunities: The following could be improved to increase the strength or speed: reducing the input resolution, reducing the number of upsampling steps, or even using a CNN-based detector in dlib, which is more pose- and lighting-invariant at the cost of extra CPU resources, as the recent comparative studies of HOG and CNN face detectors have indicated.

HOG

Histogram of Oriented Gradients (HOG) Descriptor: HOG Descriptor is based on local gradient orientations in a small area of the image and then normalizing the numbers of the histogram in two overlapping blocks before the classification. It is a design that focuses on fine scale edges, high-resolution direction binning, rough spatial aggregation, and a strict local contrast normalization, which makes it more robust to detection.

Dalal and Triggs showed that HOG descriptors along with block normalization and support vector machine (linear) are significantly more effective in pedestrian detection experiments than previous feature representations, which explains why they are effective in facial recognition experiments that rely on silhouette and contour representations that dominate at different illumination conditions.

Dlib usage: The Dlib frontal face detector was based on the paradigm of Histogram of Oriented Gradients (HOG), combining feature extraction the HOG paradigm, a linear classifier, an image pyramid, and a sliding-window search system. It offers performance with low CPU overhead that is efficient enough to work with embedded hardware and is not as resistant to large rotational perturbations as CNN-based equivalents.

Conclusion:

The current lab presents a real-time face recognition system on the Raspberry Pi which serves as a testament to a complete end-to-end computer vision system. The system includes the data capture, training of the model, and live multi-person recognition using the Pi camera and utilizing the use of the Histogram of Oriented Gradients (HOG) to extract features. The workflow highlights the importance of the careful development of datasets, strict configuration of the library, and solid environmental control which all contributed to the reliable results in terms of identification in the course of live testing, thus meeting the requirements of the established course goals. Although the system had been satisfactory with frontal and moderately angled facial orientations in most normal indoor lighting conditions, it also revealed the resource constrained nature of embedded hardware and limitations of the conventional HOG detectors when challenged by difficult poses or extreme lighting community. As a result, the project provides a realistic practical basis in computer vision systems, which outlines both the practicality of the operations and the feasibility of the practicality of implementing real time recognizing activities on low-cost platforms.

References:

- 1) N. Dalal and B. Triggs, "Histograms of Oriented Gradients for Human Detection," in Proc. CVPR, 2005. <https://lear.inrialpes.fr/people/triggs/pubs/Dalal-cvpr05.pdf>
- 2) Dlib Team, "Face Detector (HOG + Linear SVM, image pyramid, sliding window)," dlib example documentation. https://dlib.net/face_detector.py.html

Setting up Real-Time Face Recognition

- 3) CS6461--Computer Vision Systems, "Lab-3: Real-Time Face Recognition," course handout and instructions.
- 4) A. Rosebrock, "Face detection with dlib (HOG and CNN)," PyImageSearch, 2021.
<https://pyimagesearch.com/2021/04/19/face-detection-with-dlib-hog-and-cnn/>
- 5) A. K. Nath, "Face Detection with Dlib using HOG and Linear SVM," DebuggerCafe, 2024.
<https://debuggercafe.com/face-detection-with-dlib-using-hog-and-linear-svm/>
- 6) "Face recognition using deep learning on Raspberry Pi," The Computer Journal, 2024.
<https://academic.oup.com/comjnl/article/67/10/3020/7754455>